

Criptanálise de Implementação do Shamir Secret Sharing: Um Estudo de Caso do Desafio "Share" (HITCON CTF 2023)

Segurança Cibernética 2026/2 UFSCar

Bruno Camargo Ribeiro

Bruno Hiroki Nagao Anhaia

Cilene Renata Real

Emerson Hermann Lira dos Santos

Gabriel Alves Moreira

Jonathan Choy Rivera

Thayná Marostica Machado da Silva

Stephanie Maria Braga

Sorocaba

2026

Introdução

O Shamir Secret Sharing (SSS), proposto por Adi Shamir em 1979, é um esquema criptográfico fundamental para a divisão segura de segredos. O desafio "Share", do evento HITCON CTF 2023, proposto por maple3142, exige a recuperação de um segredo de 32 bytes (256 bits) protegido por este protocolo. O servidor fornece $n - 1$ partes (shares) do segredo. Sob a premissa de uma implementação perfeita, seria matematicamente impossível recuperar o segredo com essa quantidade de partes.

Este trabalho analisa a segurança de uma implementação específica do esquema de compartilhamento de segredos Shamir Secret Sharing (SSS), apresentada no desafio "Share" do HITCON CTF 2023. Explora-se a fundamentação matemática do SSS, incluindo aritmética modular, interpolação de Lagrange e o Teorema Chinês do Resto (CRT). O estudo demonstra como uma falha de implementação na geração aleatória dos coeficientes do polinômio quebra a segurança teórica do algoritmo, permitindo a recuperação total do segredo através de um ataque que utiliza o valor não gerado como testemunha de erro. Apresenta-se também uma solução otimizada em Python, eliminando a dependência de bibliotecas externas complexas.

2. Fundamentação Teórica e Matemática

2.1 Aritmética Modular

A aritmética modular é, em essência, a matemática de sistemas cíclicos. Para fins didáticos, deve-se pensar em um relógio analógico clássico de 12 horas. Se agora são 10 horas e passam-se 4 horas, o ponteiro não aponta para um inexistente "14"; ele dá a volta e para no 2. O limite do relógio (neste caso, o 12) é o que se chama de "módulo". Toda a operação criptográfica abordada neste desafio ocorre em "módulo p", sendo p um número primo grande.

Num relógio de 12 horas, depois do 12 os valores retornam ao início, ou seja, $13 = 1$ nesse sistema. Formalmente, $a \bmod n$ é o resto da divisão de a por n:

- $13 \bmod(12) = 1$ (pois $13 = 1 * 12 + 1$)
- $7 \bmod(5) = 2$ (pois $7 = 1 * 5 + 2$)
- $20 \bmod(5) = 0$ (pois $20 = 4 * 5 + 0$, divisão exata)

As operações de soma e multiplicação funcionam normalmente, apenas voltando ao começo quando ultrapassam o limite do módulo:

- $(8 + 9) \bmod(12) = 17 \bmod(12) = 5$
- $(4 * 5) \bmod(7) = 20 \bmod(7) = 6$

A divisão é o ponto que exige mais cuidado. Em aritmética modular, não existe divisão no sentido usual da matemática normal (onde dividir 5 por 2 gera o número quebrado 2,5). Em vez disso, a operação é feita multiplicando pelo inverso modular.

O inverso de um número a é um valor a^{-1} tal que $a * a^{-1} \bmod(p) = 1$ (sendo o 1 equivalente ao elemento neutro da divisão, já que não é possível escrever a/a).

- *Exemplo no módulo p = 7:* Para encontrar o inverso de 3, precisamos de um x tal que $3x \bmod(7) = 1$. A resposta é $x = 5$, porque $3 * 5 = 15$, e $15 \bmod(7) = 1$. Logo, $3^{-1} \bmod(7) = 5$.

Na prática computacional, não é necessário calcular isso manualmente e linguagens como Python (através de `pow(a, -1, p)`, disponível desde a versão 3.8) ou SageMath fazem isso diretamente.

Em criptografia, substituí-se o "12" do relógio por um número primo gigantesco. Fontes externas de matemática discreta explicam que usa-se corpos finitos (limites fechados e primos)

porque os computadores precisam calcular tudo com números inteiros exatos, garantindo que a criptografia nunca perca dados por aproximação.

Quando p é primo, todo número diferente de zero tem um inverso modular único. É por isso que algoritmos como o Shamir Secret Sharing e a interpolação de Lagrange exigem trabalhar em módulo de um primo p e isso garante que as divisões (essenciais na fórmula de Lagrange) sempre tenham uma solução exata.

2.2 Shamir Secret Sharing (SSS)

Criado em 1979 por Adi Shamir (cocriador do famoso algoritmo RSA), o *Shamir Secret Sharing* (SSS) é um protocolo que utiliza propriedades geométricas de polinômios para dividir um segredo em várias partes (*shares*).

A lógica pode ser visualizada em uma folha de papel: se tem apenas um ponto desenhado e precisa traçar a reta correta passando por ele, pode-se desenhar infinitas retas. Mas, se receber dois pontos, só existe uma única reta possível no universo que conecte os dois.

- Se você tem uma reta (grau 1), precisa de 2 pontos para desenhá-la.
- Se tem uma parábola (grau 2), precisa de 3 pontos.

Um polinômio de grau $k-1$ precisa obrigatoriamente de k pontos distintos para ser descoberto. O grau de curvatura do polinômio é definido por $k-1$, onde k é o número mínimo de partes necessárias para reconstruir o polinômio.

O algoritmo esconde a senha mestra exatamente no ponto onde a reta ou a curva cruza a parede esquerda do gráfico (o eixo Y). Ou seja, o segredo S é embutido como o termo independente do polinômio, garantindo que o segredo seja o valor exato quando $x = 0$, resultando em $f(0) = S$.

- *Exemplo Prático:* Deseja-se compartilhar o segredo $S = 42$ de forma que bastem 2 partes ($k=2$) para reconstruí-lo.

1. Gera-se um polinômio aleatório de grau 1 (uma reta): $f(x) = 42 + 7x$.
2. Os pontos são distribuídos: O Participante 1 recebe (1, 49); o Participante 2 recebe (2, 56); o Participante 3 recebe (3, 63).
3. Qualquer par desses pontos permite traçar a reta exata e descobrir que, no eixo vertical ($x=0$), o valor é 42. Um ponto isolado permitiria desenhar infinitas retas diferentes.

Pensando em um sistema de equações, um polinômio de grau $k-1$ possui k coeficientes desconhecidos (as incógnitas). Cada parte do segredo (um ponto x, y) é uma equação linear. Se um atacante ou um grupo possui apenas $k-1$ pontos (ou menos), o sistema de equações torna-se subdeterminado.

Isso significa que, com chaves insuficientes, a curva pode girar livremente para qualquer lado, tornando o $f(0)$ indescritível. Existirão infinitas curvas perfeitamente válidas que passam por aqueles pontos, e o segredo S (termo independente) pode literalmente ser qualquer número do corpo finito. Isso garante uma segurança matematicamente perfeita (*zero knowledge* — vazamento zero de informação).

Para que existam infinitas soluções de forma segura, a premissa do método é que todos os demais coeficientes do polinômio sejam escolhidos de forma 100% aleatória em todo o corpo finito. É exatamente aqui que o código do nosso desafio falha: o programador não sorteou as curvas corretamente, permitindo que o código adivinhasse para onde a curva estava apontando.

2.3 Interpolação de Lagrange

A fórmula de interpolação de Lagrange é o mecanismo matemático que descobre o polinômio inteiro cruzando os pontos conhecidos (x_i, y_i) pelas partes, extraindo o segredo $f(0)$ sem precisar deduzir um sistema de equações brutal de uma só vez.

Em vez de montar uma matriz enorme e resolver o sistema na mão para achar cada coeficiente, Lagrange calcula o resultado diretamente através de uma média ponderada específica.

A fórmula testa um ponto de cada vez: ao processar o valor do Participante X , a matemática garante que o multiplicador dele vire 1 e zere o dos demais participantes. Cada ponto contribui com um peso específico (L_i).

Matematicamente, a fórmula é expressa como:

$$f(0) = \sum y_i * L_i(0)$$

Ao somar todos os pedaços com seus respectivos pesos utilizando o inverso modular da aritmética modular em vez de divisões convencionais, as variáveis desconhecidas são canceladas, devolvendo o termo original sem nunca precisar instanciar a equação algébrica completa na memória.

Na teoria clássica, Lagrange é usado apenas para achar o $f(0)$. Contudo, no escopo do nosso script de ataque, modificamos a técnica de Lagrange para revelar todos os coeficientes da equação. Fazemos isso porque precisamos inspecionar a equação inteira para caçar a nossa

"testemunha de erro": o valor $p-1$ que o servidor do desafio nunca gera.

2.4 Teorema Chinês do Resto (CRT)

O Teorema Chinês do Resto (CRT), historicamente utilizado para contar soldados na China antiga, é um algoritmo matemático utilizado para combinar informações parciais e revelar números gigantes.

Pode-se imaginar três engrenagens de tamanhos diferentes (uma com 3 dentes, outra com 5, outra com 7) rodando conectadas a um mesmo motor. Quando se tira a venda, vê-se apenas a posição atual de cada engrenagem isolada (o que equivale ao "resto da divisão"). O CRT afirma que, apenas analisando essas posições parciais, pode-se calcular exatamente quantas milhares de voltas o motor deu.

O teorema estabelece que, se o resto da divisão de um número X por vários módulos diferentes e coprimos for conhecido, é possível reconstruir o valor exato de X em um módulo gigante, que é a multiplicação de todos os módulos individuais.

- *Exemplo Prático:* Suponha um segredo X desconhecido. Descobre-se que $X \bmod(3) = 2$ e $X \bmod(5) = 3$. Como os módulos (3 e 5) são coprimos, o CRT calcula a solução única no módulo combinado ($3 * 5 = 15$). O único número até 15 que deixa resto 2 ao dividir por 3 e resto 3 ao dividir por 5 é o número 8. Assim, o segredo original é revelado.

Na criptoanálise de falhas do SSS, cada iteração do nosso ataque contra o servidor nos fornece apenas uma visão parcial do segredo (um fragmento restringido pelo tamanho do primo p utilizado na consulta, ou seja, o *secret mod p*). O CRT é a ferramenta que costura essas múltiplas visões limitadas em uma única chave completa.

Como o nosso script só consegue quebrar o servidor e extrair o segredo modular para um primo de cada vez, nós coletamos o estado de dezenas dessas "engrenagens" (primos diferentes). Depois, o CRT alinha todas elas para revelar a chave original gigante de 256 bits. Para que essa reconstrução seja bem-sucedida, a regra matemática exige que o produto de todos os primos utilizados no ataque seja obrigatoriamente maior que o valor máximo possível do segredo (neste caso, 2^{256}).

3. Análise da Vulnerabilidade

3.1 O Código do Servidor e o Erro de Implementação

Abaixo, destaca-se o trecho crítico da classe SecretSharing original do servidor:

```
class SecretSharing:
    def __init__(self, p: int, n: int, secret: int):
```

```
self.p = p
self.n = n
self.poly = [secret] + [getRandomRange(0, self.p - 1) for _ in range(n - 1)]
```

O erro fatal ocorre na linha de geração dos coeficientes. A função `getRandomRange(a, b)` gera inteiros no intervalo que inclui a e exclui b . Portanto, `getRandomRange(0, self.p - 1)` gera números apenas no intervalo $[0, p - 2]$ e o valor $p - 1$ nunca é sorteado. Ao limitar o escopo dos coeficientes, a premissa de distribuição uniforme do SSS é quebrada, inviabilizando a segurança teórica do algoritmo.

A ausência do valor $p - 1$ é explorada como uma "testemunha de erro". O ataque consiste em arbitrar um chute para o segredo (candidato a_0) e, junto com as $n - 1$ partes recebidas, utilizar a interpolação de Lagrange para reconstruir todos os coeficientes do polinômio. Se algum coeficiente reconstruído resultar exatamente em $p - 1$, prova-se matematicamente que o chute está incorreto, pois o servidor real jamais produziria esse coeficiente. Os chutes são testados e descartados até que reste apenas um candidato válido para aquele módulo.

4. Metodologia e Contribuições do Grupo

A pesquisa caracterizou-se como um estudo de caso prático e experimental de criptoanálise aplicada. A metodologia foi dividida em quatro etapas principais: análise estática do código-fonte, preparação do ambiente computacional independente, modelagem matemática do ataque e, por fim, validação através de uma Prova de Conceito (PoC).

4.1 Análise Estática e Identificação da Vulnerabilidade

A primeira etapa consistiu em uma revisão analítica do servidor original em Python fornecido no desafio "Share". O objetivo foi contrastar a implementação prática com a definição teórica do algoritmo de *Shamir Secret Sharing* (SSS). Durante esta análise, identificou-se uma discrepância na função `getRandomRange(0, self.p - 1)`, responsável pela geração dos coeficientes do polinômio. Constatou-se um erro de *off-by-one* (erro de limite por um), resultando em um viés estatístico onde o coeficiente de valor $p - 1$ nunca é gerado. Esta descoberta formou a base vetorial do ataque.

4.2 Preparação do Ambiente Computacional

Para a execução do ataque, optou-se por descartar a abordagem original que exigia a instalação do framework matemático SageMath e a utilização de containers Docker. O ambiente foi restrito estritamente à linguagem Python (versão 3.8 ou superior), sem dependências de bibliotecas externas de terceiros.

Para viabilizar essa independência, a equipe reimplementou funções matemáticas

fundamentais do zero:

- A operação de divisão em corpos finitos foi adaptada utilizando a função nativa $\text{pow}(a, -1, p)$ para o cálculo do inverso modular.
- Foi desenvolvido um teste de primalidade de Miller-Rabin próprio para a geração dinâmica dos números primos necessários para o ataque, eliminando a necessidade de bibliotecas auxiliares como SymPy.

4.3 Modelagem e Algoritmo de Exploração

O ataque foi estruturado em um script iterativo de força bruta lógica (desenvolvido nos arquivos `demo_share_attack.py` e `demo_share_attack_texto.py`), operando na seguinte sequência de passos:

1. **Geração de Primos:** O script seleciona automaticamente um conjunto de números primos coprimos, garantindo que o produto de todos eles seja superior ao limite do segredo (256 bits).
2. **Interação Simulada:** Para cada primo p , são solicitadas $n - 1$ partes (*shares*) da função local que simula o servidor vulnerável.
3. **Força-Bruta Modular:** O script itera sobre todos os valores possíveis para o termo independente (candidato a segredo, variando de 0 até $p-1$).
4. **Filtragem por Testemunha de Erro:** Para cada candidato a segredo, o script aplica a Interpolação de Lagrange adaptada para reconstruir a totalidade dos coeficientes daquele polinômio. Se a reconstrução apresentar o valor $p - 1$ em qualquer coeficiente, o candidato é descartado matematicamente como inválido.
5. **Reconstrução via CRT:** O único candidato sobrevivente em cada módulo é armazenado. Em seguida, o algoritmo aplica o Teorema Chinês do Resto (CRT) para fundir todos os resultados modulares parciais, revelando o segredo completo.

O artefato principal gerado pelo grupo, `demo_share_attack.py`, divide-se em componentes lógicos:

- **SecretSharing e servidor_local():** Simulação local e precisa do servidor, contendo o bug original, evitando a necessidade de infraestrutura de rede.
- **lagrange_interpola_completo():** Implementação customizada de Lagrange que recupera todos os coeficientes do polinômio, essencial para a busca da testemunha $p - 1$.
- **recupera_secret_mod_p() e crt():** Núcleo lógico que realiza a força-bruta local filtrando pela testemunha de erro e junta os resultados via Teorema Chinês do Resto.

Foi desenvolvido um script adicional, `demo_share_attack_texto.py`, que eleva o ataque a um cenário de teste real. Ele implementa um teste de primalidade de Miller-Rabin totalmente do zero (dispensando bibliotecas como SymPy), converte textos reais para inteiros e calcula automaticamente a quantidade de primos necessários para exceder o tamanho do segredo.

4.4 Validação e Coleta de Métricas

Para comprovar a escalabilidade e a eficácia da implementação independente, a Prova de Conceito foi testada sob condições progressivas de carga. Foram realizados experimentos de recuperação utilizando entradas de texto arbitrárias e calculando o tempo exato para a extração ponta a ponta. A validação final consistiu em aplicar o ataque contra um alvo do mesmo tamanho exigido pelo desafio original (segredo de 32 bytes / 256 bits) para registrar o número de consultas, a quantidade de primos processados e o tempo de execução total, comprovando a viabilidade operacional do método desenvolvido.

4.5 Ambiente sem Dependências

A resolução de referência dependia do software SageMath e de um ambiente Docker com requisições de rede. A principal contribuição do grupo foi simplificar e reimplementar a prova de conceito inteiramente em Python 3 puro. Toda a aritmética modular e a interpolação de Lagrange foram codificadas do zero, utilizando nativamente a função $\text{pow}(a, -1, p)$, eliminando o peso e a complexidade de configuração do SageMath.

5. Evidência de Reprodução

A execução de ponta a ponta comprova a eficácia do método. Ao rodar o script contra um segredo de teste, o atacante recebe a validação módulo a módulo:

```
Segredo real (para conferencia): 123456789
Primos escolhidos: [17, 19, 23, 29, 31, 37, 41]
Produto dos primos: 10131543907

p=17:
Início: 17 candidatos possíveis -> [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16]
Rodada 1: eliminou 10 candidato(s) [2, 3, 4, 5, 6, 8, 10, 11, 13, 15] | sobraram 7 -> [0, 1, 7, 9, 12, 14, 16]
Rodada 2: eliminou 3 candidato(s) [0, 7, 16] | sobraram 4 -> [1, 9, 12, 14]
Rodada 3: eliminou 2 candidato(s) [9, 12] | sobraram 2 -> [1, 14]
Rodada 4: eliminou 0 candidato(s) [] | sobraram 2 -> [1, 14]
Rodada 5: eliminou 0 candidato(s) [] | sobraram 2 -> [1, 14]
Rodada 6: eliminou 0 candidato(s) [] | sobraram 2 -> [1, 14]
Rodada 7: eliminou 0 candidato(s) [] | sobraram 2 -> [1, 14]
Rodada 8: eliminou 1 candidato(s) [14] | sobraram 1 -> [1]
-> secret mod p = 1 (esperado 1) [OK] (8 consulta(s))
```

```

Reconstruindo o segredo via CRT:
  M (produto de todos os primos): 10131543907
  p= 17: resto= 1 | M_i=M/17=595973171 | inverso de M_i mod 17 = 3 | contribuicao = 1787919513
  p= 19: resto= 14 | M_i=M/19=533239153 | inverso de M_i mod 19 = 7 | contribuicao = 1599717459
  p= 23: resto= 11 | M_i=M/23=440501909 | inverso de M_i mod 23 = 11 | contribuicao = 2643011454
  p= 29: resto= 19 | M_i=M/29=349363583 | inverso de M_i mod 29 = 10 | contribuicao = 5589817328
  p= 31: resto= 2 | M_i=M/31=326823997 | inverso de M_i mod 31 = 19 | contribuicao = 2287767979
  p= 37: resto= 36 | M_i=M/37=273825511 | inverso de M_i mod 37 = 35 | contribuicao = 547651022
  p= 41: resto= 8 | M_i=M/41=247110827 | inverso de M_i mod 41 = 3 | contribuicao = 5930659848
  soma de todas as contribuicoes, mod M: 123456789

Segredo reconstruido via CRT: 123456789
Segredo real:                  123456789

```

Para comprovar a viabilidade contra o tamanho real exigido pelo desafio, foram realizados benchmarks de escalabilidade descritos na tabela abaixo, atestando que a ausência do SageMath não prejudica o desempenho do ataque em escala de CTF:

Entrada	Tamanho em Bits	Primos Necessários	Tempo de Execução (Local)
"senha de teste 2026" (20 chars)	151 bits	26 primos	~5,4s
Segredo Original do Desafio (32 bytes)	256 bits	~40 primos	~17-18s

6. Conclusão

A resolução do desafio "Share" do HITCON CTF 2023 demonstra perfeitamente uma máxima fundamental da segurança cibernética: algoritmos matematicamente provados como inquebráveis falham quando mal implementados no código. Embora o *Shamir Secret Sharing* (SSS) garanta uma segurança teórica incondicional apoiada pela robustez da aritmética modular, a vulnerabilidade do software ao não gerar coeficientes polinomiais de forma estritamente aleatória, a omissão do valor $p-1$, invalidou a premissa de "vazamento zero" do protocolo. Ao combinar a Interpolação de Lagrange, utilizada não para seu fim original, mas para expor as equações defeituosas, com o Teorema Chinês do Resto (CRT), capaz de costurar esses pequenos vazamentos de dados em uma chave completa de 256 bits, a anatomia deste ataque comprova que a resiliência criptográfica não reside apenas no teórico, mas exige uma execução computacional rigorosa e totalmente isenta de vieses estruturais na geração de aleatoriedade.

7. Referências Bibliográficas

- **0XATTICUS**. [HITCON 2023] *Share write-up*. Disponível em: https://www.0xatticus.com/posts/hitcon2023/hitcon2023_share/. Acesso em: 02 set. 2026.
- **FRAGMENTIX**. *What is Secret Sharing?* Disponível em: <https://fragmentix.com/what-is-secret-sharing/>. Acesso em: 02 set. 2026.
- **NIST - National Institute of Standards and Technology**. *NIST Digital Library of Mathematical Functions: Interpolation (Seção 3.3)*. Disponível em: <https://dlmf.nist.gov/3.3>. Acesso em: 02 set. 2026.
- **SHAMIR, Adi**. How to Share a Secret. *Communications of the ACM*, v. 22, n. 11, p. 612–613, nov. 1979. DOI: 10.1145/359168.359176. Disponível em: <https://dl.acm.org/doi/10.1145/359168.359176>. Acesso em: 02 set. 2026.
- **WEISSTEIN, Eric W**. Chinese Remainder Theorem. *Wolfram MathWorld*. Disponível em: <https://mathworld.wolfram.com/ChineseRemainderTheorem.html>. Acesso em: 02 set. 2026.
- **WIKIPEDIA**. *Shamir's Secret Sharing*. Disponível em: https://en.wikipedia.org/wiki/Shamir%27s_secret_sharing. Acesso em: 02 set. 2026.

Repositórios, Ferramentas e Metodologia

- **ANTHROPIC**. *Claude (Inteligência Artificial)*. Utilizado como ferramenta de apoio metodológico para: (1) reimplementação local da classe *SecretSharing* original em Python (preservando a lógica e a vulnerabilidade do servidor); e (2) conversão da solução criptográfica de SageMath para Python puro (funções *lagrange_interpolacao_completo*, *poly_mul* e teste de primalidade de Miller-Rabin). 2026.
- **CRYPTOHACK**. *CTF Archive - HITCONCTF-2023-Share*. Cópia preservada do desafio original contendo o Dockerfile para simulação do ambiente.
- **MAPLE3142**. *HITCON CTF 2023 / Share*. Código-fonte e repositório original do autor do desafio.